Sections and Chapters

Anton Nørgaard

August 3, 2025

Contents

0.1 Introduction

Chapter 1

Prerequisites for compiling and running the game

This chapter details all the necessary requirements for setting up, preparing and running the game.

It wort noting that the game can **only** run on Linux operating systems as it uses certain C-libraries that only work in Unix-like operating systems.

Furthermore, at the risk of sounding arrogant and snarky, this guide assumes you are quite competent with programming, Linux and computers in general.

Getting the correct compiler

The game was and should be compiled using GCC. To my recollection, it uses GCC-specific, non c-standard features, so it is unlikely it will work otherwise.

All of this of course, assumes you have sufficient privileges to install software on the desired host.

Installing GCC via Ubuntu

Firstly, run

```
sudo apt update
```

To update the packages list. Next, run

```
sudo apt install build-essential
```

This actually installs a group of different tools, most of which you will need anyway to compile the project, like make as well.

To verify GCC has been installed, run

```
gcc --version
```

This should print out the version of GCC in use.

Additional required software

Installing ncurses

The code uses the ncurses library to draw all the characters on the screen. It is therefore necessary in order to compile the code.

Installing ncurses on systems with apt

You need to install two packages

1. libncurses5-dev (developer's libraries for ncurses)

2. libncursesw5-dev (developer's libraries for ncursesw)

To install both, run

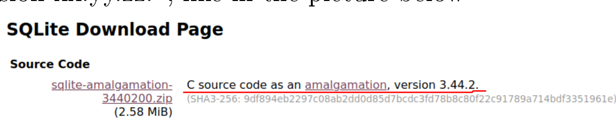
```
sudo apt-get install libncurses5-dev libncursesw5-dev
```

Installing SQLite

Revenant also uses SQLite to manage game state. Furthermore, it needs several other components for allowing compilation itself. There are actually several valid approaches for integrating, compiling and running SQLite, but this particular setup is based on what at the time seemed the best.

Getting the SQLite code

To get the SQLite source code, go to `https://www.sqlite.org/download.html` and download whatever is specified to be "C source code as an amalgamation, version xx.yy.zz.", like in the picture below



Alternatively, assuming you have wget and you know the version number(in the case 3.44.2), you could also run

```
wget https://www.sqlite.org/2023/sqlite-amalgamation-3440200.zip
```

Then unzip the downloaded file.

Compiling the CLI interface

The game uses the command-line interface to generate the game world and the like. To enable the command line, go into the directory

```
gcc shell.c sqlite3.c -lpthread -ldl -lm -o sqlite3
```

This

Getting,compiling and generating the actual game

Now that everything has been set up, the time has come to actually get, compile and generate the game world. To do this, simply go in to the revenant/misc folder and run the following shell script

```
sudo bash ./setup_gamle_world.sh
```

and then promptly sit back and relax. Chances are, this is going to take a while.

Chapter 2

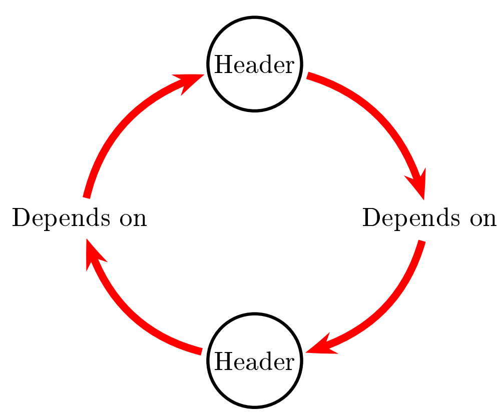
Source code documentation

The purpose of this chapter is two-fold

1. It intends to explain to the player the logic behind how certain things work, for educational purposes.
2. The chapter will also serve as a personal specification / written in stone manual for how certain things are done, to ensure things are done consistently.

Setup of header files and forward declaration and the db_reader

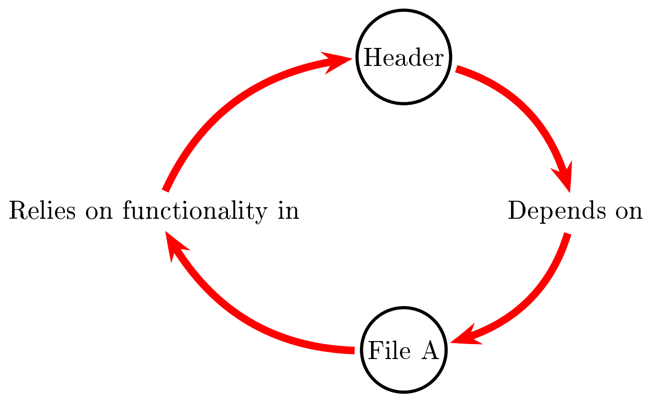
A recurring issue in the code base as coding continued and complexity increased, was the often almost spontaneous discovery of circular dependencies - declarations (header files) that would depend on each other:



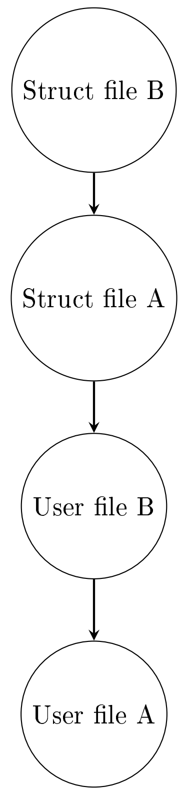
The solution to this issue was to hyper split the header files in the following manner, containing the following information

- Struct header files. These would include **only** struct declarations and constants.
- "User" header files. These files would in turn define functions and other actual code functionality. Said files would in one way or another make use of the struct header files, hence the classification as users

The idea behind this principle is that it would address the primary issue that caused said circular dependencies - namely that functionality in file A would rely on some functionality in file B that would do some sort of manipulation of structs for file A, that are defined in file A, hence causing said circular dependency:

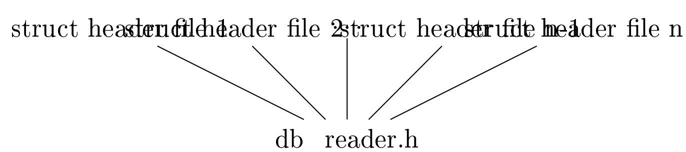


With the new setup (having separate header files for structs and functionality that relies and manipulates structs), this will no longer happen. This is because with the new setup, the "inheritance", going back to the example with the A and B files, will work as follows



By separating the definitions of structs and the logic that uses the structs, we have managed to "break" the circular definitions. It is admittedly unknown whether this will cause further headaches down the line, but it succeeds in solving the original issue, which makes it an acceptable solution for now.

Which brings us to



2.0.1 Policy on when to include header files

When a header file is to be included in either a source code file or a header file i.e when we make a statement of the type

```
#include "some_header.h"
```

Then it is to be included only *exactly* the momenet when it become relevant, that is to say, when the code first refers to functionality/a declaration, made in the header it is including.

This is because

1. It makes it transparent why the include statement was made.
2. Furthermore, avoids cluttering with unecessary include statements, potentially causing compiling errors and makes it more easy in reasoning what has been defined in which parts of the code.

Naming conventions for the header files

Naming conventions for the files themselves

As specified in "Setup of header files and forward declaration and the db_reader ", there are header files that define only structs and constants and user files, that in some way or another uses said structs. They shall have the name the following name formats

- Header files that use structs can have whatever name they so desire, as long as they are explanatory
- Header files, that only define the structs, must end with the _struct.h, to indicate that it only defines structs and other constants

Naming for functions, structs and macros in the files, along with the files themselves

Any and all functions/macros/structs, that are defined in the same header file, must begin with the same prefix, e.g if a header file defines 10 functions and the chosen prefix is say, abc, each and every function must begin with abc_, the prefix should ideally be a descriptive acronym of the intended functionality of the header file. Mixed upper and lower case are permitted and it it is not required to be consistent, whether the prefix is in uppercase or lower case.

This rule applies for both the struct header files and the user header files i.e if the user header file inherits from the struct header file, they must have the same prefix acronym, to indicate the inheritance or that the user file uses the structs defined in the struct file (see Setup of header files and forward declaration and the db_reader) for more information.

When it comes to the actual naming of the files themselves, the prefix of the file in turn decides the prefix of the various objects in the header/source files.

Management of game state and status

Keeping track of the creatures and the player

Checking for active effects of player

Creatures

In the following chapters, we will use several variables to compute various information regarding creatures in-game, their status and the way that they interact with the environment and the player. For that purpose, we will use the following definitions, some are explicitly computed/stored in-game, others are just definitions and are not explicitly declared in-game. The definitions we will use are

- min_local the smallest local coordinate value possible
- max_local the biggest local coordinate value possible
- $Player_{global\ coordinate}$ to represent a player's global coordinate, either X or Y coordinate
- $Creature_{global\ coordinate}$ to represent a creature's global coordinate, either X or Y coordinate
- The exact same variables are defined for the local coordinates
- max_moves is the maximum number of tiles a player/creature can move, before they move our of bounds of the current screen view. It is computed as (max_local - max_local) $+1$
- $Player_{set\ to\ min}$ is a subtraction of the players global coordinate s.t its local coordinate is the minimal. It is computed as $Player_{global\ coordinate}$ - ($Player_{local\ coordinate}$ - min_local)
- $Player_{set\ to\ max}$ is an addition of the players global coordinate s.t its local coordinate is the maximal. It is computed as $Player_{global\ coordinate}$ $+$ (max_local - $Player_{local\ coordinate}$)

With that in mind, we can now go over the different mechanics that we need to consider for the creatures in game, such as spawning, movement, death, behavior, actions and so on

Spawning creatures

When it comes to spawning creatures, there are several things that we need to compute in relation to the player. One of them is the creatures local coordinates. The kicker about this is that while a player's local coordinates can be arbitrary without any hassle, the coordinates of the creature must be aligned relative to the players to that the view the creature has on screen matches that of the global coordinates. To do this, there are two cases that need to be considered, the trivial case and the nontrivial case.

In the trivial case, the distance between the player and the creature is s.t the distance for the particular axis, does not exceed the boundaries of the screen

In the non-trivial case, what we effectively do is reduce it to the nontrivial case. The way that we do this is by considering whether the axis-wise coordinate for the player is greater or smaller than the creature's just like in the non-trivial case. In the case that

$$Player_{global\ coordinate} > Creature_{global\ coordinate}$$

Then set the creature's local coordinate to

$$Creature_{local\ coordinate} = max\_local - (Player_{set\ to\ max} - Creature_{global\ coordinate}) \mod max\_moves$$

In the other case where

$$Player_{global\ coordinate} < Creature_{global\ coordinate}$$

Instead, set the creature's local coordinate to

$$Creature_{local\ coordinate} = min\_local + (Player_{set\ to\ min} - Creature_{global\ coordinate}) \mod max\_moves$$

The workings of this are a little hard to work around. Admittedly, this took rather a long while to figure out and I am unsure whether the computations can get simplified further. However, truth be told, I spent way too long to figure this out and so at this point that I cannot be bothered to come up with more clever solutions.

The way it works (informally) is as follows. Assume that

Controls

Keyboard Layout

This table details all the keyboard bindings for the game. Keyboard bindings with an asterisk indicate that

a-m	n-z	A-M	N-Z+ESC
a - N/A	n - N/A	A - N/A	N - N/A
b - N/A	o - N/A	B - N/A	O - N/A
c - N/A	p - N/A	C - N/A	P - N/A
d - N/A	q - quit game / context	D - N/A	Q - N/A
e - equip item in inventory	r - N/A	E - Show equipped items	R - N/A
f - N/A	s - N/A	F - N/A	S - N/A
g - N/A	t - N/A	G - N/A	T - N/A
h - N/A	u - N/A	H - N/A	U - N/A
i - Display current inventory	v - N/A	I - N/A	W - N/A
j - N/A	w - N/A	J - N/A	X- N/A
k - N/A	x - N/A	K - N/A	Y - N/A
l - show past 10 events	y - N/A	L - N/A	Z - N/A
m - N/A	z - N/A	M - N/A	ESC - Quit command

Table 2.1: Keyboard bindings.

a-m	n-z	A-M	N-Z+ESC	Misc.
a - animal	n - N/A	A - N/A	N - N/A	@ - player character
b - N/A	o - N/A	B - N/A	O - N/A	! - interactable npc
c - N/A	p - N/A	C - N/A	P - N/A	
d - N/A	q - N/A	D - N/A	Q - N/A	
e - N/A	r - N/A	E - N/A	R - N/A	
f - N/A	s - N/A	F - N/A	S - N/A	
g - N/A	t - trader	G - N/A	T - N/A	
h - N/A	u - N/A	H - N/A	U - N/A	
i - N/A	v - N/A	I - N/A	W - N/A	
j - N/A	w - N/A	J - N/A	X- N/A	
k - N/A	x - N/A	K - N/A	Y - N/A	
l - N/A	y - N/A	L - N/A	Z - N/A	
m - N/A	z - N/A	M - N/A	ESC - Quit command	

Table 2.2: Meaning of characters on screen.

Character meanings on map

Interacting with the world

Dialogue

Dialogue screen and interaction, dialogue-wise

As unintuitive as it is, dialogue options are **0-indexed**. This is because it gives a maximum of 10 possible options (0-9), without needing logic for a buffer and instead just accept a single character as player input. As of time of this writing, it is hoped it is never required to have more than 10 options, because it means, as said, we can skimp on buffer logic.

Handling of picking dialogue options flow and the subsequent design of the database

There are to be defined a total of at least 3 tables

- dia_dialogue_interactions - intended to correspond almost directly to a Dia_Dialogue_Manager manager, minus meta data about the current state of the dialogue session such as weather the player has reached the end of the current dialogue file, weather the file fits into a single window etc.
- dia_dialogue_selected_option_handler - inteded to manage the overall flow of updating the dialogue. When a player selects an option, it will be used to decide what the next dialogue id should be (if any), what checks are to be made, if any and the next dialogue id's provided the player fails or suceeds the checks respectively.
- dia_dialogue_option_triggers defines the set of triggers to fire

The general flow of the dialogue flow is to be as follows

1. Assuming the player picked a valid dialogue option, query dia_dialogue_selected_option_handler. In this table, it will check what kind of check to do, if any, to decide what the next dialogue file should be. If the check is to simply continue dialogue, just read in what the next dialogue flag is set to be which will be stored in the success check flag value. If the check is to exit the dialogue, store the exit flag in the current dialogue id, then when the game loop sees, that the current dialogue id is actually an exit flag, it will quit the dialogue session. Provided a check is defined, the success or failure is decided by another query in the current callback for the query to dia_dialogue_selected_option_handler. The result of the check is in the relevant callback stored in the query manager struct.
2. In conunction

Design choices for the dialogue flow

This dialogue / database design opens op for a hefty bit of redundancy in the dia_dialogue_selected_option_handler table - in that it stores both the id for the next dialogue file if a check is succesfull and if the check fails, but the failure id will be redundant if there is no check (the dialogue trivially progresses to the next dialogue id) and both will be redundant if there is no check and no next dialogue id (selecting said option culminates in exiting dialogue) But the alternative would be to split it into yet more tables - requireing more indexes, more queries, more logic and more overall management. Hence, we opted to introduce a fair amount of redundancy to skimp design / coding logic.

It has by design been decided that when it comes to dynamically deciding what the next dialogue id should be read in, there can only be at most one binary check. So it's not possible for there to be multiple checks to decide what the next dialoge id should be and it is to be decided by a successfull check or a failure. The motivation for this is choice is that otherwise, the logical complexity for maintaining all the different checks would be mind boggling to follow and troubleshoot, given they're stored and retried in SQL queries.¨

Managing, reading, writing and manipulation of the game state

The database

Naming standards

Naming standard for variables

Naming of Macros In addition to the naming standards specified in section 2.0.1, the naming standards for the database components adheres to the following set of rules and conventions.

Since many values, especially in the context of dialogues are integer id based, and the sqlite interface uses indexes to refer to the values in the columns of the database, the game makes extensive use of macros, to make it clear what it is we are referring to and to reduce the likelihood of errors when index accessing .

Macros that refer to values stored in the columns have two naming formats. This is due to an inconsistency of the sqlite approach for how variables are numbered. When binding variables for queries, the leftmost variable has and index of **1**, but when retrieving values from a query, the leftmost value has an index of **0**.

Macros that refer column values in the context of a SELECT, UPDATE, DELETE, etc., have the name format of

dbr_<table name>_<column name>INDEX_QUERY

And macros that refer to extracting column values, from a row as a result of a SELECT have the name format of

dbr_<table name>_<column name>INDEX_QRESULT

Macros that indicate id's of ingame objects (creatures,npc's, items, etc) are to be named as dbr_<table name>_<OBJECT NAME>_ID

NOTE! Per standard defined in "Naming for functions and macros in the files", variables must also be prefixed with dbr_

Naming conventions for the db_reader.c & db_reader.h files All structs, that represent or or more value(s) derived from a query, shall be named *_Qresult, to indicate it is the result of a query, analogous to the naming convention for the macros.

If there is a 1-to-1 correspondence between the value(s) returned from a query and a desired struct in said context, the dbr_reader is allowed to return said struct directly, rather than a *_Qresult struct.

Naming standard for actual database objects The naming standard for the database objects like tables, triggers, etc. Follow the rules defined in section 2.0.1.

Placement and order of declaration

Declaration order for database components To ensure readability and ease of information finding, any and all declaration of database objects are made in the misc/ game folder, using shell scripts.

The database objects are to be declared in this order

1. The table itself
2. Any triggers, indexes, constraints, etc.
3. The values in the table

Each new declaration is to be separated using a dividing line, made from ###########

At convenience and as needed, each declaration can have elaborating comments

Declaration order for the db_reader.c & db_reader.h By design, the db_reader functionality will *not* include header information from other revenant components. Instead, any components the need information read in from the DB will include db_reader in its header files. By doing so, it reduces db_reader's dependency on the fields contained in the structs of the components that use db_reader, allowing it to be agnostic wrt. the type and amount of information returned from the queries.

Database schema

Column 1	Column 2	Column 3
A1	B1	C1
Merged Columns		C3
D1	D2	D3
All Columns Merged		
F1	G1	H1

In this subsection, we will list all the

Table:dia_dialogue_interactions			
Column name	Column index	Data type	Description
dialogue_folder_id	0	int	The id of the dialogue folder
current_dialogue_id	1	int	A dialogue file in the dialogue folder in question. Identified by an id
num_dialogue_options	2	int	The number of dialogue options available for the dialogue_folder_id + current_dialogue_id
Indexes, keys and constraints			
PRIMARY KEY(dialogue_folder_id, current_dialogue_id) CREATE UNIQUE INDEX IF NOT EXISTS(dialogue_folder_id, current_dialogue_id)			
Dependencies of dia_dialogue_interactions			
Table dependencies of dia_dialogue_interactions			
Table name			Role
dia_dialogue_selected_option_consequence_handler - **??**			If any checks are to be made as a consequence of a selected dialogue options, e.g skill check, trigger check this table will also have to be read from.

Table:dia_dialogue_selected_option_handler			
Column name	Column index	Data type	Description
dialogue_folder_id	0	int	dialogue folder id of the Dia_Dialogue_Manager used for the current dialogue session.
current_dialogue_id	1	int	The current dialogue_id file being read from the dialogue folder.
selected_option	2	int	The selected dialogue option for the current dialogue_folder_id and current_dialogue_id context
check_type	3	int	What type of check to commence
next_dia_id_if_check_success	4	int	The next current dialogue to read in, provided the passed the checks, if any.
next_dia_id_if_check_fail	5	int	The next current dialogue to read in, provided the passed the checks, if any.
Indexes, keys and constraints			
PRIMARY KEY(dialogue_folder_id, current_dialogue_id, selected_option CREATE UNIQUE INDEX IF NOT EXISTS(dialogue_folder_id, current_dialogue_id, selected_option			
FOREIGN KEY(dialogue_folder_id, current_dialogue_id), REFERENCES dia_dialogue_interactions(dialogue_folder_id, current_dialogue_id)			
Table name			Role
selected_dialogue_option			If any checks are to be made as a consequence of a selected dialogue options, e.g skill check, trigger check this table will also have to be read from.

<table>
<tr><th colspan="4">Table:dia_dialogue_option_triggers</th></tr>
<tr><td>Column name</td><td>Column index</td><td>Data type</td><td>Description</td></tr>
<tr><td>Column name</td><td>Column index</td><td>Data type</td><td>Description</td></tr>
<tr><td>dialogue_folder_id</td><td>0</td><td>int</td><td>dialogue folder id of the Dia_Dialogue_Manager used for the current dialogue session.</td></tr>
<tr><td>current_dialogue_id</td><td>1</td><td>int</td><td>The current dialogue_id file being read from the dialogue folder.</td></tr>
<tr><td>selected_option</td><td>2</td><td>int</td><td>The selected dialogue option for the current dialogue_folder_id and current_dialogue_id context</td></tr>
<tr><td>on_outcome</td><td>3</td><td>int</td><td>Is the trigger applied on a success or failure.</td></tr>
<tr><td>trigger_type</td><td>4</td><td>int</td><td>what type of trigger to be applied, based on the selected choice of the player</td></tr>
<tr><th colspan="4">Indexes, keys and constraints</th></tr>
<tr><td colspan="4">CREATE UNIQUE INDEX IF NOT EXISTS(dialogue_folder_id, current_dialogue_id, selected_option</td></tr>
<tr><td colspan="4">FOREIGN KEY(dialogue_folder_id, current_dialogue_id), REFERENCES dia_dialogue_interactions(dialogue_folder_id, current_dialogue_id)</td></tr>
<tr><td colspan="3">Table name</td><td>Role</td></tr>
<tr><td colspan="3">selected_dialogue_option</td><td>If any checks are to be made as a consequence of a selected dialogue options, e.g skill check, trigger check this table will also have to be read from.</td></tr>
</table>

Table:dia_trigger_set_initial_dialogue			
Column name	Column index	Data type	Description
dialogue_folder_id	0	int	dialogue folder id of the Dia_Dialogue_Manager used for the current dialogue session.
current_dialogue_id	1	int	The current dialogue_id file being read from the dialogue folder.
selected_option	2	int	The selected dialogue option for the current dialogue_folder_id and current_dialogue_id context
on_outcome	3	int	Is the initial id set based on the success or failure of the check.
initial_id	4	int	The next initial id
Indexes, keys and constraints			
PRIMARY KEY(dialogue_folder_id, current_dialogue_id, selected_option, on_outcome CREATE UNIQUE INDEX IF NOT EXISTS(dialogue_folder_id, current_dialogue_id, selected_option, on_outcome			
FOREIGN KEY(dialogue_folder_id, current_dialogue_id), REFERENCES dia_dialogue_interactions(dialogue_folder_id, current_dialogue_id)			
Table name			Role
selected_dialogue_option			If any checks are to be made as a consequence of a selected dialogue options, e.g skill check, trigger check this table will also have to be read from.

The world of Revenant

Places of Revenant

Settlements

Iislog

An incredibly modest

Who you are